МИНОБРНАУКИ РОССИИ
федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технологический университет «СТАНКИН» (ФГАОУ ВО «МГТУ «СТАНКИН»)

Институт информационных технологий

Кафедра управления и информатики в технических системах

ОТЧЕТ О ВЫПОЛНЕНИИ ЛАБОРАТОРНОЙ РАБОТЫ № 1 ПО ДИСЦИПЛИНЕ

«Прикладное программирование»

СТУДЕНТА *2* КУРСА *бакалавриата* ГРУППЫ *ИДБ-25-12*
(уровень профессионального образования)

Крылов Станислав Павлович

НА ТЕМУ

ООП в Python. Обработка исключительных ситуаций.
Форматы XML и JSON

Направление: 09.03.03 Прикладная информатика
Профиль подготовки: Управление данными

Отчет сдан «_____»_____20___г.

Оценка _____

Преподаватель Гетманов С. В. _____
(Ф.И.О., должность, степень, звание.) *(подпись)*

МОСКВА 2026

СОДЕРЖАНИЕ

1 Цель работы 3

2 Задание 3

3 Ход работы 3

3.1 Описание предметной области . . . 3

3.2 Диаграмма классов . . . 3

3.3 Описание классов . . . 4

3.4 Обработка исключений . . . 4

3.5 Структура хранения данных в JSON и XML . . . 5

3.6 Реализация CRUD, чтения и записи файлов . . . 5

3.7 Пользовательский интерфейс . . . 5

3.8 Работа с системой контроля версий . . . 6

4 Вывод 6

Приложение А. Листинг программного кода 7

Файл models.py . . . 7

Файл exceptions.py . . . 8

Файл service.py . . . 8

Файл storage.py . . . 9

Файл main.py . . . 11

1 Цель работы

Изучить объектно-ориентированное программирование на Python на примере предметной области текстового редактора, построить диаграмму классов, обработать встроенные и собственные исключения, реализовать хранение данных в форматах XML и JSON.

2 Задание

Вариант 16: написать классы для предметной области «Текстовый редактор». Необходимо создать диаграмму классов, реализовать приложение в объектно-ориентированном стиле, предусмотреть обработку исключений и файловый обмен в форматах XML и JSON.

3 Ход работы

3.1 Описание предметной области

Программа хранит набор документов. Каждый документ имеет уникальный идентификатор, название и упорядоченный список текстовых блоков. Поддерживаются абзац, заголовок с уровнем от 1 до 6, цитата и элемент маркированного списка. Различные блоки по-разному отображают текст через общий метод render().

Пользователь может создавать, просматривать, переименовывать и удалять документы; добавлять, просматривать, изменять и удалять блоки; искать фрагмент текста без учёта регистра. Данные автоматически сохраняются в рабочем JSON-файле и могут экспортироваться или импортироваться в JSON и XML.

Модуль	Назначение
models.py	Документ, текстовые блоки и результат поиска.
service.py	Каталог документов и операции редактора.
exceptions.py	Иерархия собственных исключений.
storage.py	Чтение и запись JSON/XML.
main.py	Командная строка и точка входа.

3.2 Диаграмма классов

Диаграмма подготовлена в PlantUML и хранится в файле docs/classes.puml. На ней показаны наследование видов блоков от TextBlock, связь документа со множеством блоков, связь каталога с документами, зависимости редактора и хранилищ, а также иерархия исключений.

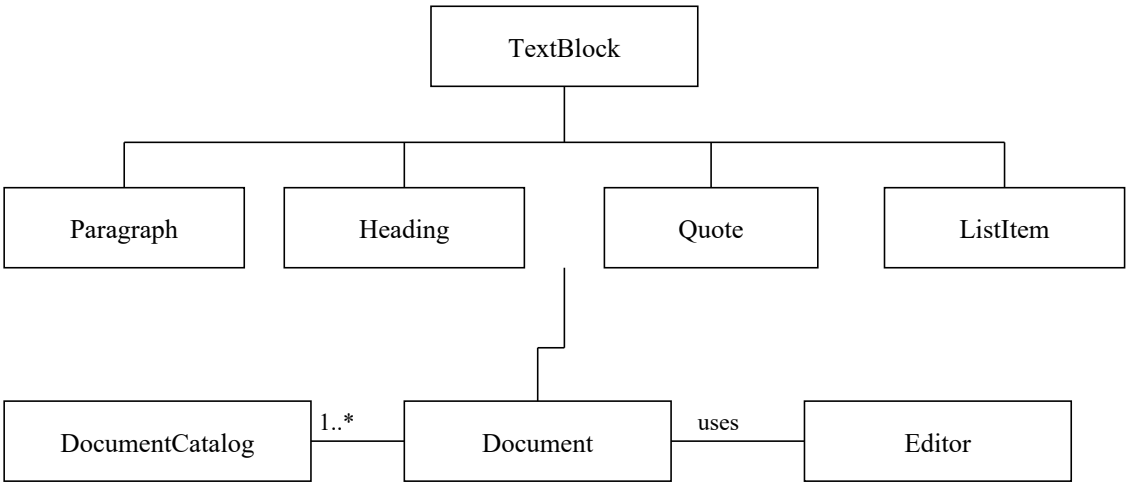


Рисунок 1 — Основные связи классов текстового редактора. Полная диаграмма, включая Storage и исключения, приложена в редактируемом файле docs/classes.puml.

3.3 Описание классов

Класс	Назначение
TextBlock	Базовый блок с текстом и идентификатором.
Paragraph	Абзац без дополнительного форматирования.
Heading	Заголовок с проверяемым уровнем 1–6.
Quote	Цитата с префиксом >.
ListItem	Элемент маркированного списка.
Document	Название, ID и список блоков.
SearchResult	Неизменяемый результат поиска.
DocumentCatalog	Хранение документов и CRUD.
Editor	CRUD блоков и поиск.
Storage	Абстрактный интерфейс чтения и записи.
JsonStorage	Сериализация и загрузка JSON.
XmlStorage	Сериализация и загрузка XML.
EditorCLI	Разбор и выполнение консольных команд.

Наследование реализовано в семействе TextBlock и хранилищ Storage. Полиморфизм проявляется при вызове render() у разных блоков и load()/save() у разных хранилищ. Инкапсуляция используется в DocumentCatalog: словарь документов хранится в поле _documents, а доступ к нему осуществляется через методы. Для полей, функций и методов указаны аннотации типов.

3.4 Обработка исключений

Все собственные ошибки наследуются от EditorError. ValidationError сообщает о неверном названии, тексте или уровне заголовка; NotFoundError — об отсутствующем документе или блоке; ConflictError — о повторяющемся ID; FormatError — о некорректной структуре файла либо неподдерживаемом расширении.

При чтении JSON перехватывается json.JSONDecodeError, при чтении XML — xml.etree.ElementTree.ParseError. Ошибки KeyError и ValueError в структуре данных преобразуются в FormatError. На уровне CLI также перехватываются OSError и UnicodeError. Сообщение выводится в stderr, команда завершается с кодом 1; некорректный импорт не перезаписывает текущий каталог.

3.5 Структура хранения данных в JSON и XML

В JSON корневой объект содержит массив documents. Каждый документ содержит id, title и массив blocks. Блок записывается с id, kind и text; заголовок дополнительно содержит level. Порядок блоков сохраняется. При XML-сериализации используется корневой элемент documents и вложенные элементы document и block. Специальные символы экранируются стандартной библиотекой.

```
{
  "documents": [{
    "id": "example1", "title": "Пример документа",
    "blocks": [{"id": "head1", "kind": "heading",
                "text": "Мой текст", "level": 1}]
  }]
}
```

```
<?xml version="1.0" encoding="utf-8"?>
<documents>
  <document id="example1" title="Пример документа">
    <block id="head1" kind="heading" level="1">Мой текст</block>
  </document>
</documents>
```

3.6 Реализация CRUD, чтения и записи файлов

Операция	Документы	Блоки
Create	DocumentCatalog.add	Editor.add_block
Read	DocumentCatalog.get/list	Editor.get_block
Update	DocumentCatalog.rename	Editor.update_block
Delete	DocumentCatalog.delete	Editor.delete_block

DocumentCatalog хранит документы в словаре по ID; Editor работает с блоками выбранного документа. Классы JsonStorage и XmlStorage реализуют интерфейс Storage. При загрузке данные преобразуются обратно в экземпляры Document и соответствующих подклассов TextBlock. Проверяются структура, типы, уровни заголовков и уникальность ID.

3.7 Пользовательский интерфейс

Консольное приложение запускается командой python main.py. Команды new, list, show, rename, delete обеспечивают работу с документами; add, edit, remove — с блоками; search выполняет поиск; import и export обмениваются файлами. Рабочая база по умолчанию —

data/editor.json. Пример запуска:

```
python main.py import data/sample.json
python main.py show example1
python main.py add example1 heading "Новый раздел" --level 2
python main.py search редактор
python main.py export data/backup.xml
```

Изменения после каждой команды сохраняются автоматически. Параметр --db позволяет выбрать другой рабочий файл JSON. Программа не требует сторонних пакетов.

3.8 Работа с системой контроля версий

Проект ведётся в Git. В истории сохранены отдельные коммиты для реализации редактора, добавления документации и упрощения структуры. Репозиторий: https://github.com/stanislavkrylov2007-cell/text-editor-lab. Файлы рабочей базы и временные файлы исключены через .gitignore; демонстрационный data/sample.json включён в проект.

4 Вывод

В ходе лабораторной работы по варианту 16 создан текстовый редактор на Python. Спроектирована UML-диаграмма с классами предметной области и исключениями; реализованы наследование, полиморфизм, инкапсуляция, CRUD документов и блоков, поиск, валидация и обработка ошибок. Данные сохраняются и восстанавливаются в JSON и XML. Консольный интерфейс и файловый обмен проверены на примере документа.

Приложение А. Листинг программного кода

Файл models.py

```
"""Документы и виды текстовых блоков."""

from __future__ import annotations

from dataclasses import dataclass, field
from uuid import uuid4

from exceptions import ValidationError


@dataclass
class TextBlock:
    text: str
    id: str = field(default_factory=lambda: uuid4().hex[:8])
    kind: str = field(default="paragraph", init=False)

    def __post_init__(self) -> None:
        if not isinstance(self.text, str) or not isinstance(self.id, str) or not self.id:
            raise ValidationError("Текст должен быть строкой, ID — непустой строкой")

    def render(self) -> str:
        return self.text


@dataclass
class Paragraph(TextBlock):
    kind: str = field(default="paragraph", init=False)


@dataclass
class Heading(TextBlock):
    level: int = 1
    kind: str = field(default="heading", init=False)

    def __post_init__(self) -> None:
        super().__post_init__()
        if type(self.level) is not int or not 1 <= self.level <= 6:
            raise ValidationError("Уровень заголовка должен быть от 1 до 6")

    def render(self) -> str:
        return f"{'#' * self.level} {self.text}"


@dataclass
class Quote(TextBlock):
    kind: str = field(default="quote", init=False)

    def render(self) -> str:
        return f"> {self.text}"


@dataclass
class ListItem(TextBlock):
    kind: str = field(default="list_item", init=False)

    def render(self) -> str:
        return f"- {self.text}"


BLOCK_TYPES: dict[str, type[TextBlock]] = {
    "paragraph": Paragraph, "heading": Heading,
    "quote": Quote, "list_item": ListItem,
}

@dataclass
class Document:
    title: str
    id: str = field(default_factory=lambda: uuid4().hex[:8])
    blocks: list[TextBlock] = field(default_factory=list)

    def __post_init__(self) -> None:
        if not isinstance(self.title, str) or not self.title.strip():
            raise ValidationError("Название документа не может быть пустым")
```

```
        if not isinstance(self.id, str) or not self.id:
            raise ValidationError("ID документа не может быть пустым")

    def render(self) -> str:
        return "\n\n".join(block.render() for block in self.blocks)


@dataclass(frozen=True)
class SearchResult:
    document_id: str
    block_id: str
    text: str
```

Файл exceptions.py

```
"""Исключения текстового редактора."""

class EditorError(Exception):
    """Базовая ошибка редактора."""


class ValidationError(EditorError):
    """Некорректные данные пользователя."""


class NotFoundError(EditorError):
    """Документ или блок не найден."""


class FormatError(EditorError):
    """Некорректный формат файла."""


class ConflictError(EditorError):
    """Идентификатор уже существует."""
```

Файл service.py

```
"""Операции над документами и блоками."""

from __future__ import annotations

from exceptions import ConflictError, NotFoundError, ValidationError
from models import Document, SearchResult, TextBlock


class DocumentCatalog:
    def __init__(self, documents: list[Document] | None = None) -> None:
        self._documents: dict[str, Document] = {}
        for document in documents or []:
            self.add(document)

    def add(self, document: Document) -> None:
        if document.id in self._documents:
            raise ConflictError(f"Документ {document.id} уже существует")
        self._documents[document.id] = document

    def get(self, document_id: str) -> Document:
        try:
            return self._documents[document_id]
        except KeyError as exc:
            raise NotFoundError(f"Документ {document_id} не найден") from exc

    def list(self) -> list[Document]:
        return list(self._documents.values())

    def rename(self, document_id: str, title: str) -> Document:
        document = self.get(document_id)
        if not title.strip():
            raise ValidationError("Название документа не может быть пустым")
        document.title = title
        return document

    def delete(self, document_id: str) -> None:
        self.get(document_id)
        del self._documents[document_id]
```

```
class Editor:
    def __init__(self, catalog: DocumentCatalog) -> None:
        self.catalog = catalog

    def add_block(self, document_id: str, block: TextBlock) -> None:
        document = self.catalog.get(document_id)
        if any(item.id == block.id for item in document.blocks):
            raise ConflictError(f"Блок {block.id} уже существует")
        document.blocks.append(block)

    def get_block(self, document_id: str, block_id: str) -> TextBlock:
        for block in self.catalog.get(document_id).blocks:
            if block.id == block_id:
                return block
        raise NotFoundError(f"Блок {block_id} не найден")

    def update_block(self, document_id: str, block_id: str, text: str) -> TextBlock:
        if not isinstance(text, str):
            raise ValidationError("Текст должен быть строкой")
        block = self.get_block(document_id, block_id)
        block.text = text
        return block

    def delete_block(self, document_id: str, block_id: str) -> None:

        document = self.catalog.get(document_id)
        block = self.get_block(document_id, block_id)
        document.blocks.remove(block)

    def search(self, query: str) -> list[SearchResult]:
        if not query:
            raise ValidationError("Поисковый запрос не может быть пустым")
        return [SearchResult(doc.id, block.id, block.text)
                for doc in self.catalog.list() for block in doc.blocks
                if query.casefold() in block.text.casefold()]
```

Файл storage.py

```
"""Чтение и запись каталога в JSON и XML."""

from __future__ import annotations

import json
from abc import ABC, abstractmethod
from pathlib import Path
from xml.etree import ElementTree as ET

from exceptions import FormatError, ValidationError
from models import BLOCK_TYPES, Document, TextBlock
from service import DocumentCatalog


def block_to_dict(block: TextBlock) -> dict:
    result = {"id": block.id, "kind": block.kind, "text": block.text}
    if block.kind == "heading":
        result["level"] = block.level
    return result


def catalog_to_dict(catalog: DocumentCatalog) -> dict:
    return {"documents": [{"id": doc.id, "title": doc.title,
                           "blocks": [block_to_dict(block) for block in doc.blocks]}
                          for doc in catalog.list()]}


def catalog_from_dict(data: dict) -> DocumentCatalog:
    try:
        if not isinstance(data, dict) or not isinstance(data["documents"], list):
            raise FormatError("Ожидался объект с массивом documents")
        documents = []
        for item in data["documents"]:
            if not isinstance(item["blocks"], list):
                raise FormatError("blocks должен быть массивом")
            blocks = []
            for raw in item["blocks"]:
                cls = BLOCK_TYPES[raw["kind"]]
                args = {"id": raw["id"], "text": raw["text"]}
                if cls is BLOCK_TYPES["heading"]:
```

```
                    args["level"] = raw["level"]
                blocks.append(cls(**args))
            ids = [block.id for block in blocks]
            if len(ids) != len(set(ids)):
                raise FormatError("Повторяющийся ID блока")
            documents.append(Document(item["title"], item["id"], blocks))
        return DocumentCatalog(documents)
    except (KeyError, TypeError, ValueError, ValidationError) as exc:
        raise FormatError(f"Некорректная структура данных: {exc}") from exc


class Storage(ABC):
    @abstractmethod
    def load(self, path: Path) -> DocumentCatalog:
        pass

    @abstractmethod
    def save(self, path: Path, catalog: DocumentCatalog) -> None:
        pass


class JsonStorage(Storage):
    def load(self, path: Path) -> DocumentCatalog:
        try:
            return catalog_from_dict(json.loads(path.read_text(encoding="utf-8")))
        except json.JSONDecodeError as exc:
            raise FormatError(f"Некорректный JSON: {exc}") from exc

    def save(self, path: Path, catalog: DocumentCatalog) -> None:
        path.parent.mkdir(parents=True, exist_ok=True)
        path.write_text(json.dumps(catalog_to_dict(catalog), ensure_ascii=False,
                                   indent=2) + "\n", encoding="utf-8")


class XmlStorage(Storage):
    def load(self, path: Path) -> DocumentCatalog:
        try:
            root = ET.parse(path).getroot()
            if root.tag != "documents":
                raise FormatError("Корневой элемент должен быть documents")
            data = {"documents": []}
            for doc in root:
                if doc.tag != "document":
                    raise FormatError("Ожидался элемент document")
                item = {"id": doc.attrib["id"], "title": doc.attrib["title"],
                        "blocks": []}
                for block in doc:
                    if block.tag != "block":
                        raise FormatError("Ожидался элемент block")
                    raw = {"id": block.attrib["id"], "kind": block.attrib["kind"],
                           "text": block.text or ""}
                    if raw["kind"] == "heading":
                        raw["level"] = int(block.attrib["level"])
                    item["blocks"].append(raw)
                data["documents"].append(item)
            return catalog_from_dict(data)
        except (ET.ParseError, KeyError, ValueError) as exc:
            raise FormatError(f"Некорректный XML: {exc}") from exc

    def save(self, path: Path, catalog: DocumentCatalog) -> None:
        root = ET.Element("documents")
        for doc in catalog.list():
            element = ET.SubElement(root, "document", id=doc.id, title=doc.title)
            for block in doc.blocks:
                attrs = {"id": block.id, "kind": block.kind}
                if block.kind == "heading":
                    attrs["level"] = str(block.level)
                ET.SubElement(element, "block", **attrs).text = block.text
        ET.indent(root, space="  ")
        path.parent.mkdir(parents=True, exist_ok=True)
        ET.ElementTree(root).write(path, encoding="utf-8", xml_declaration=True)


def storage_for(path: Path) -> Storage:
    if path.suffix.lower() == ".json":
        return JsonStorage()
    if path.suffix.lower() == ".xml":
```

```
        return XmlStorage()
    raise FormatError("Поддерживаются только .json и .xml")
```

Файл main.py

```
"""Консольный интерфейс. Изменения автоматически сохраняются в базе JSON."""

from __future__ import annotations

import argparse
import sys
from pathlib import Path

from exceptions import EditorError
from models import BLOCK_TYPES, Document, Heading
from service import DocumentCatalog, Editor
from storage import JsonStorage, storage_for


class EditorCLI:
    def __init__(self) -> None:
        self.parser = argparse.ArgumentParser(description="Текстовый редактор, вариант 16")
        self.parser.add_argument("--db", type=Path, default=Path("data/editor.json"),
                                 help="Путь к рабочей базе JSON")
        commands = self.parser.add_subparsers(dest="command", required=True)
        new = commands.add_parser("new", help="Создать документ")
        new.add_argument("title")
        commands.add_parser("list", help="Список документов")
        show = commands.add_parser("show", help="Показать документ и ID блоков")
        show.add_argument("document_id")
        rename = commands.add_parser("rename", help="Переименовать документ")
        rename.add_argument("document_id")
        rename.add_argument("title")
        delete = commands.add_parser("delete", help="Удалить документ")
        delete.add_argument("document_id")
        add = commands.add_parser("add", help="Добавить текстовый блок")
        add.add_argument("document_id")
        add.add_argument("kind", choices=BLOCK_TYPES)
        add.add_argument("text")
        add.add_argument("--level", type=int, default=1)
        edit = commands.add_parser("edit", help="Изменить текст блока")
        edit.add_argument("document_id")
        edit.add_argument("block_id")
        edit.add_argument("text")
        remove = commands.add_parser("remove", help="Удалить блок")
        remove.add_argument("document_id")
        remove.add_argument("block_id")
        search = commands.add_parser("search", help="Поиск по блокам")
        search.add_argument("query")
        export = commands.add_parser("export", help="Экспорт в JSON/XML")
        export.add_argument("path", type=Path)
        imported = commands.add_parser("import", help="Импорт JSON/XML вместо текущей
базы")
        imported.add_argument("path", type=Path)

    def run(self, argv: list[str] | None = None) -> int:
        args = self.parser.parse_args(argv)
        try:
            if args.command == "import":
                catalog = storage_for(args.path).load(args.path)
                JsonStorage().save(args.db, catalog)
                print(f"Импортировано документов: {len(catalog.list())}")
                return 0
            catalog = JsonStorage().load(args.db) if args.db.exists() else
DocumentCatalog()
            editor = Editor(catalog)
            changed = False
            if args.command == "new":
                doc = Document(args.title)
                catalog.add(doc)
                print(f"Создан документ {doc.id}: {doc.title}")

                changed = True
            elif args.command == "list":
                for doc in catalog.list():
                    print(f"{doc.id}\t{doc.title}\t{len(doc.blocks)} блоков")
            elif args.command == "show":
                doc = catalog.get(args.document_id)
```

```
                print(f"{doc.title} [{doc.id}]")
                for block in doc.blocks:
                    print(f"[{block.id}] {block.render()}")
            elif args.command == "rename":
                catalog.rename(args.document_id, args.title)
                changed = True
            elif args.command == "delete":
                catalog.delete(args.document_id)
                changed = True
            elif args.command == "add":
                cls = BLOCK_TYPES[args.kind]
                block = Heading(args.text, level=args.level) if cls is Heading else
cls(args.text)
                editor.add_block(args.document_id, block)
                print(f"Создан блок {block.id}")
                changed = True
            elif args.command == "edit":
                editor.update_block(args.document_id, args.block_id, args.text)
                changed = True
            elif args.command == "remove":
                editor.delete_block(args.document_id, args.block_id)
                changed = True
            elif args.command == "search":
                for result in editor.search(args.query):
                    print(f"{result.document_id}/{result.block_id}\t{result.text}")
            elif args.command == "export":
                storage_for(args.path).save(args.path, catalog)
                print(f"Экспортировано: {args.path}")
            if changed:
                JsonStorage().save(args.db, catalog)
            return 0
        except (EditorError, OSError, UnicodeError) as exc:
            print(f"Ошибка: {exc}", file=sys.stderr)
            return 1


def main(argv: list[str] | None = None) -> int:
    return EditorCLI().run(argv)


if __name__ == "__main__":
    raise SystemExit(main())
```